



Mata Kuliah : PASD
Program Studi : D4 – Sistem Informasi Bisnis
Semester : 2

Kelas : 1F
NIM : 254107060124
Nama : Muhammad Khairibyan Athallah
Jobsheet Ke- : 11

Laporan Jobsheet

Praktikum Ke-1

Langkah	Praktikum 2.1 — Pembuatan Single Linked List:
1	<pre>public class Mahasiswa15 { String nim; String nama; String kelas; double ipk; public Mahasiswa15() { } public Mahasiswa15(String nm, String name, String kls, double ip) { this.nim = nm; this.nama = name; this.kelas = kls; this.ipk = ip; } public void tampilInformasi() { System.out.printf(format: "%-10s %-12s %-5s %.1f%n", nama, nim, kelas, ipk); } }</pre>
2	<pre>J NodeMahasiswa15.java > NodeMahasiswa15 > NodeMahasiswa15(Mahasiswa15, NodeMahasiswa15) 1 public class NodeMahasiswa15 { 2 Mahasiswa15 data; 3 NodeMahasiswa15 next; 4 5 public NodeMahasiswa15(Mahasiswa15 data, NodeMahasiswa15 next) { 6 this.data = data; 7 this.next = next; 8 } 9 } 10</pre>

Praktikum Ke-2

Langkah	Praktikum 2.1 — Pembuatan Single Linked List:
1	<pre>J SingleLinkedList15.java > SingleLinkedList15 > print() 1 public class SingleLinkedList15 { 2 NodeMahasiswa15 head; 3 NodeMahasiswa15 tail; 4 5 boolean isEmpty() { 6 return (head == null); 7 } 8 9 public void print() { 10 if (!isEmpty()) { 11 NodeMahasiswa15 tmp = head; 12 System.out.println(x: "Isi Linked List:"); 13 while (tmp != null) { 14 tmp.data.tampilInformasi(); 15 tmp = tmp.next; 16 } 17 System.out.println(x: ""); 18 } else { 19 System.out.println(x: "Linked list kosong"); 20 } 21 } 22 23 public void addFirst(Mahasiswa15 input) { 24 NodeMahasiswa15 ndInput = new NodeMahasiswa15(input, next: null); 25 if (isEmpty()) { 26 head = ndInput; 27 tail = ndInput; 28 } else { 29 ndInput.next = head; 30 head = ndInput; 31 } 32 } 33 34 public void addLast(Mahasiswa15 input) { 35 NodeMahasiswa15 ndInput = new NodeMahasiswa15(input, next: null); 36 if (isEmpty()) { 37 head = ndInput;</pre>

```
SingleLinkedList15.java / SingleLinkedList15 / print()
1 public class SingleLinkedList15 {
34 public void addLast(Mahasiswa15 input) {
35     NodeMahasiswa15 ndInput = new NodeMahasiswa15(input, next: null);
36     if (isEmpty()) {
37         head = ndInput;
38         tail = ndInput;
39     } else {
40         tail.next = ndInput;
41         tail = ndInput;
42     }
43 }
44
45 public void insertAfter(String key, Mahasiswa15 input) {
46     NodeMahasiswa15 ndInput = new NodeMahasiswa15(input, next: null);
47     NodeMahasiswa15 temp = head;
48     do {
49         if (temp.data.nama.equalsIgnoreCase(key)) {
50             ndInput.next = temp.next;
51             temp.next = ndInput;
52             if (ndInput.next == null) {
53                 tail = ndInput;
54             }
55             break;
56         }
57         temp = temp.next;
58     } while (temp != null);
59 }
60
61 public void insertAt(int index, Mahasiswa15 input) {
62     if (index < 0) {
63         System.out.println(x: "indeks salah");
64     } else if (index == 0) {
65         addFirst(input);
66     } else {
67         NodeMahasiswa15 temp = head;
68         for (int i = 0; i < index - 1; i++) {
69             temp = temp.next;
70         }
71         temp.next = new NodeMahasiswa15(input, temp.next);
72         if (temp.next.next == null) {
73             tail = temp.next;
74         }
75     }
76 }
```

Praktikum Ke-3

Langka h	Praktikum 2.1 — Pembuatan Single Linked List:	
1	<pre> 33 34 public void addLast(Mahasiswa15 input) { 35 NodeMahasiswa15 ndInput = new NodeMahasiswa15(input, next: null); 36 } if (isEmpty()) { 37 head = ndInput; 38 tail = ndInput; 39 } else { 40 tail.next = ndInput; 41 tail = ndInput; 42 } 43 } 44 45 public void insertAfter(String key, Mahasiswa15 input) { 46 NodeMahasiswa15 ndInput = new NodeMahasiswa15(input, next: null); 47 NodeMahasiswa15 temp = head; 48 do { 49 if (temp.data.nama.equalsIgnoreCase(key)) { 50 ndInput.next = temp.next; 51 temp.next = ndInput; 52 if (ndInput.next == null) { 53 tail = ndInput; 54 } 55 break; 56 } 57 temp = temp.next; 58 } while (temp != null); 59 } 60 61 public void insertAt(int index, Mahasiswa15 input) { 62 if (index < 0) { 63 System.out.println("indeks salah"); 64 } else if (index == 0) { 65 addFirst(input); 66 } else { 67 NodeMahasiswa15 temp = head; 68 for (int i = 0; i < index - 1; i++) { 69 temp = temp.next; 70 } 71 temp.next = new NodeMahasiswa15(input, temp.next); 72 if (temp.next.next == null) { 73 tail = temp.next; 74 } 75 } 76 } </pre>	
2	Pertanyaan 2.1.2 1. Mengapa hasil compile kode program di baris pertama menghasilkan "Linked List Kosong"? 2. Jelaskan kegunaan variable temp secara umum pada setiap method! 3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard! Jawaban : 1. Karena saat sll.print() pertama kali dipanggil, belum ada data yang dimasukkan ke dalam linked list. Variabel head masih bernilai null, sehingga method isEmpty() mengembalikan true dan program langsung mencetak "Linked list kosong". 2. Variable temp digunakan sebagai penunjuk sementara untuk menelusuri (traverse) node-node dalam linked list satu per satu dari head sampai node terakhir, tanpa mengubah atau menggeser posisi head aslinya. Jika langsung menggunakan head untuk traversal, maka referensi ke node pertama akan hilang dan linked list tidak bisa diakses lagi dari awal.	

```

System.out.println(x: "Tambah data dari keyboard:");
System.out.print(s: "Masukkan NIM   : ");
String nim = sc.nextLine();
System.out.print(s: "Masukkan Nama : ");
String nama = sc.nextLine();
System.out.print(s: "Masukkan Kelas : ");
String kelas = sc.nextLine();
System.out.print(s: "Masukkan IPK   : ");
double ipk = sc.nextDouble();
sc.nextLine();
Mahasiswa15 mhs = new Mahasiswa15(nim, nama, kelas, ipk);
sll.addLast(mhs);
sll.print();

sc.close();
}

```

3.

```

public void getData(int index) {
    NodeMahasiswa15 tmp = head;
    for (int i = 0; i < index; i++) {
        tmp = tmp.next;
    }
    tmp.data.tampilInformasi();
}

public int indexOf(String key) {
    NodeMahasiswa15 tmp = head;
    int index = 0;
    while (tmp != null && !tmp.data.nama.equalsIgnoreCase(key)) {
        tmp = tmp.next;
        index++;
    }

    if (tmp == null) {
        return -1;
    } else {
        return index;
    }
}

public void removeFirst() {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih Kosong, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
    }
}

public void removeLast() {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih Kosong, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        NodeMahasiswa15 temp = head;
        while (temp.next != tail) {
            temp = temp.next;
        }
        temp.next = null;
        tail = temp;
    }
}

```

```

    }

    public void remove(String key) {
        if (isEmpty()) {
            System.out.println(x: "Linked List masih Kosong, tidak dapat dihapus!");
        } else {
            NodeMahasiswa15 temp = head;
            while (temp != null) {
                if ((temp.data.nama.equalsIgnoreCase(key)) && (temp == head)) {
                    this.removeFirst();
                    break;
                } else if (temp.data.nama.equalsIgnoreCase(key)) {
                    temp.next = temp.next.next;
                    if (temp.next == null) {
                        tail = temp;
                    }
                    break;
                }
                temp = temp.next;
            }
        }
    }

    public void removeAt(int index) {
        if (index == 0) {
            removeFirst();
        } else {
            NodeMahasiswa15 temp = head;
            for (int i = 0; i < index - 1; i++) {
                temp = temp.next;
            }
            temp.next = temp.next.next;
            if (temp.next == null) {
                tail = temp;
            }
        }
    }
}

```

Pertanyaan 2.2.3

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!

2. Jelaskan kegunaan kode berikut pada method remove:

```
temp.next = temp.next.next;
```

```
if (temp.next == null) {
    tail = temp;
}
```

jawaban :

1. Keyword break digunakan untuk menghentikan perulangan while setelah node yang dicari berhasil ditemukan dan dihapus. Karena tujuan method remove hanya menghapus satu node yang cocok dengan key, maka tidak perlu melanjutkan penelusuran ke node berikutnya. Tanpa break, program akan terus berjalan dan bisa menyebabkan error karena pointer linked list sudah berubah setelah penghapusan.

2. temp.next = temp.next.next : memotong node dari linked list dengan cara melewatinya langsung, sehingga node tersebut tidak lagi terhubung dan dianggap terhapus.

if (temp.next == null) { tail = temp; } : mengecek apakah node yang dihapus adalah node terakhir. Jika iya, maka tail diperbarui ke temp agar tail tetap menunjuk ke node paling akhir yang benar.

--	--	--

tugas

Langkah	Jawaban/Deskripsi
1	
2	

